

Manual Técnico de Modelos de Machine Learning

Este manual técnico describe los modelos de machine learning implementados, incluyendo su configuración, estructuras de datos de entrada, parámetros utilizados, y una explicación detallada de su funcionamiento.

Índice

1. [Introducción](#)
 2. [Estructura de Datos](#)
 3. [Modelos Implementados](#)
 - [Regresión Lineal](#)
 - [Regresión Polinómica](#)
 - [Árbol de Decisión](#)
 - [Red Neuronal](#)
 - [K-Means](#)
 - [K-Nearest Neighbor](#)
 4. [Parseo del CSV](#)
 5. [Errores Comunes y Soluciones](#)
 6. [Recomendaciones](#)
-

Introducción

Este documento detalla el funcionamiento de una serie de modelos de machine learning implementados en un sistema. Cada modelo se elige según el tipo de problema que se desea resolver, y el procesamiento de datos está adaptado para cada uno. Los modelos permiten realizar predicciones y clasificaciones basadas en diferentes algoritmos de machine learning, tales como regresión, clustering y clasificación.

Estructura de Datos

Para todos los modelos, los datos se cargan mediante un archivo CSV que se analiza y procesa antes de ser utilizados. El CSV está estructurado de la siguiente manera:

```
Entrenamiento;Punto;Euclidean;Manhattan  
"A,9,8,8,I;B,2,1,2,III;C,9,8,9,I;D,8,9,9,II;E,1,8,2,I;F,9,8,8,III;G,2,2,3,II";"  
H,5,7,9";"II";"I,II"
```

Cada campo representa lo siguiente:

- **Entrenamiento:** Datos de entrenamiento separados por comas.
- **Punto:** Punto específico para clasificar o predecir.

- **Euclideano:** Tipo de métrica o configuración para el cálculo.
- **Manhattan:** Otra métrica o configuración.

Modelos Implementados

Regresión Lineal

Descripción

El modelo de regresión lineal encuentra una relación lineal entre una variable dependiente y una o más variables independientes.

Estructura de Datos

- **Entrada:** Dos columnas de datos, x (variable independiente) y y (variable dependiente).
- **Salida:** Coeficientes de la recta de mejor ajuste.

Parámetros

- **xTrain:** Valores de x utilizados para el entrenamiento.
- **yTrain:** Valores de y correspondientes para el entrenamiento.

Código de Implementación

```
if (modelType == "linear-regression") {
  lines.forEach(line => {
    const [x, y] = line.split(';').map(parseFloat);
    xTrain.push(x);
    yTrain.push(y);
  });
}
```

Regresión Polinómica

Descripción

La regresión polinómica es una extensión de la regresión lineal que permite modelar relaciones de orden superior entre las variables.

Estructura de Datos

- **Entrada:** Columnas de x, y, y xPred.
- **Salida:** Coeficientes del polinomio de ajuste.

Parámetros

- **xTrain:** Valores de x para el entrenamiento.
- **yTrain:** Valores de y para el entrenamiento.
- **xToPredict:** Valores de x sobre los que se harán predicciones.

Código de Implementación

```

if (modelType == "polynomial-regression") {
  lines.forEach(line => {
    const [x, y, xPred] = line.split(';').map(parseFloat);
    xTrain.push(x);
    yTrain.push(y);
    xToPredict.push(xPred);
  });
}

```

Árbol de Decisión

Descripción

El modelo de árbol de decisión realiza clasificaciones y predicciones mediante un árbol jerárquico de decisiones, basado en la estructura de los datos.

Estructura de Datos

- **Entrada:** Varias columnas representando atributos y etiquetas de clase.
- **Salida:** Estructura de árbol para realizar clasificaciones.

Parámetros

- **headers:** Nombres de los atributos en la primera fila del CSV.
- **xTrain:** Datos de entrenamiento, incluyendo atributos y etiquetas de clase.

Código de Implementación

```

if (modelType === "decision-tree") {
  headers = lines[0].split(','); // Cabeceras de atributos
  const trainData = [];
  lines.slice(1).forEach(line => {
    const attributes = line.split(',').map(item => item.trim());
    trainData.push(attributes);
  });
  xTrain = [headers, ...trainData];
}

```

Red Neuronal

Descripción

La red neuronal es un modelo de machine learning inspirado en el cerebro humano, adecuado para problemas complejos y de gran dimensión.

Estructura de Datos

- **Entrada:** Valores numéricos.

- **Salida:** Predicciones o clasificaciones basadas en las capas de la red.

Parámetros

- **arrNR:** Array de valores numéricos.

Código de Implementación

```
if (modelType == "neuronal-network") {  
    arrNR = lines[0].split(',').map(Number); // Convierte cada valor a entero  
}
```

K-Means

Descripción

K-means es un algoritmo de clustering que agrupa los datos en k clusters basándose en la distancia euclidiana.

Estructura de Datos

- **Entrada:** Número de clusters, datos de entrenamiento, número de iteraciones.
- **Salida:** Clusters con datos agrupados.

Parámetros

- **clusters:** Número de clusters especificado.
- **trainingData:** Datos de entrenamiento para el clustering.
- **iterations:** Número de iteraciones.

Código de Implementación

```
if (modelType === "k-means-linear") {  
    lines.forEach(line => {  
        const [numeroClusters, entrenamiento, numeroIteraciones] =  
line.split(';');  
        clusters = parseInt(numeroClusters, 10);  
        trainingData = entrenamiento.split(',').map(Number);  
        iterations = parseInt(numeroIteraciones, 10);  
    });  
}
```

K-Nearest Neighbor

Descripción

K-Nearest Neighbor (KNN) es un algoritmo de clasificación que clasifica nuevos datos en función de los k puntos de datos más cercanos.

Estructura de Datos

- **Entrada:** Datos de entrenamiento, punto de predicción, métricas.
- **Salida:** Clasificación o predicción basada en los vecinos cercanos.

Parámetros

- **entrenamientoValue:** Datos de entrenamiento en formato CSV.
- **puntoValue:** Punto a clasificar.
- **euclideanValue:** Métrica euclidiana para KNN.
- **manhattanValue:** Métrica Manhattan para KNN.

Código de Implementación

```
if (modelType === "k-nearest-neighbor") {  
    const [entrenamiento, punto, euclidean, manhattan] =  
lines[0].split(';').map(item => item.trim());  
    const entrenamientoValue = entrenamiento.replace(/^"$/g, '');  
    const puntoValue = punto.replace(/^"$/g, '');  
    const euclideanValue = euclidean.replace(/^"$/g, '');  
    const manhattanValue = manhattan.replace(/^"$/g, '');  
}
```

Parseo del CSV

La función de parseo permite cargar y procesar los datos en función del tipo de modelo. La función se asegura de extraer correctamente cada valor del CSV y prepararlo para su uso en el modelo correspondiente.

```
function parseCSVData(data) {  
    // Lógica de parseo según el tipo de modelo  
}
```

Errores Comunes y Soluciones

1. **Datos en Formato Incorrecto:** Verificar el formato del CSV, ya que un error común es la falta de comillas en los valores.
2. **Número de Parámetros Insuficiente:** Cada modelo requiere un conjunto específico de datos, asegúrate de que cada campo esté correctamente poblado.
3. **Error de Tipo de Dato:** Usar `parseFloat` o `parseInt` cuando sea necesario para convertir datos numéricos.

Recomendaciones

1. **Validación de Datos:** Implementa funciones de validación antes de procesar el CSV.

2. **Logging:** Utiliza console.log o herramientas de logging para verificar que los datos se procesen correctamente.
3. **Documentación Continua:** Actualiza este manual a medida que se agreguen o modifiquen modelos.